The System-3/0 Blueprint: A Formally Verified Orchestration Framework for Multi-Agent Systems

Jyotirmoy Singh

Birla Institute of Technology and Science, Pilani

singhjyotirmoy17@gmail.com

1. Supervisor Dr. Anand Rao

Heinz College of Information Systems and Public Policy

Carnegie Mellon University

2. Supervisor Dr. Chittaranjan Hota

Department of Computer Science and Information Systems

BITS Pilani, Hyderabad Campus

Abstract

Current Multi-Agent System (MAS) frameworks fail in production for three recurring reasons: they spend expensive models on simple tasks, they let malformed outputs break downstream steps, and they have weak stopping control in long reasoning loops. We present the System-3/0 Blueprint to address those issues at the architecture level, not at the prompt level.

The design splits orchestration from execution. System-3 handles planning, routing, and budget control. System-0 is a deterministic contract boundary that enforces structured outputs through constrained decoding and schema validation. System-1 agents are fast specialists with tool access. System-2 agents are deliberate reasoners with no direct tools; they call tools only through System-1. This design keeps tool interactions auditable and stops structural errors from propagating through multi-step pipelines.

The main systems contribution is a dynamic Agent Registry, $\mathcal{R}$: `AgentID` $\rightarrow$ `AgentSpec`, where each agent is defined by a typed 6-tuple $\langle M_i, G_i, K_i, V_i, C_i, T_i \rangle$. The main formal contribution comprises six theorems, a lemma, and two design arguments covering safety, routing, and necessity: constrained decoding safety and depth-k composability, optimal routing under bounded resources, sublinear regret in contextual allocation, and formal limits of monolithic reasoning policies.

The framework is validated through a paired ablation study across two conditions (300 runs, 150 per condition). When the System-3/0 architecture is removed (Baseline condition), the Cascading Failure Rate rises from 0.0% to 38.0% ($p < 0.001$, Cohen's $h = -1.328$), directly confirming the depth-k composability proof. The System 0/3 condition simultaneously reduces mean token expenditure by 3.6$\times$ (1,600 vs 5,724 tokens, $p < 0.001$). The reference implementation runs locally on consumer hardware (24 GB RAM, no GPU) via Ollama (`phi3:mini` / `llama3.1:8b`), with no external API dependency. The reference implementation is publicly available at `https://github.com/Jyotirmoy17/System-3-0-MAS-OS`.

1 Introduction

1.1 The Deployment Problem in Multi-Agent Systems

Current Multi-Agent System (MAS) architectures fail in production for systems reasons, not because base models are too weak. In repeated deployments, three failure modes appear together:

- Cost Unpredictability: Regardless of task complexity, using frontier reasoning models for all operations results in unbounded and unsustainable compute expenditure.
- Cascading Fragility: In the absence of formal output contracts, a single malformed JSON payload or syntactic error generated by an upstream agent silently corrupts the execution context of all downstream agents.
- Non-Terminating Reasoning: Without a clear mathematical stopping criterion, autonomous loops (like ReAct) have a tendency to spiral into arbitrarily expensive cycles, depleting their budget while arguing with themselves.

These are systems problems, so they need systems controls. Our central claim is that MAS should be treated as a typed computational architecture with explicit orchestration guarantees, not as loosely coupled prompt chains.

1.2 The Architectural Gap: The Reasoning-Resource Paradox

Current frameworks rely on the model to self-regulate its tool use and halting circumstances, delegating orchestration to the LLM itself. We provide formal evidence that this method is theoretically unsound. The Reasoning-Resource Paradox states that a single monolithic strategy cannot simultaneously optimise for resource conservation and solution accuracy, as formalised in Design Argument 3.2. The agent

will inevitably display either a “Laziness Bias” (halting instantly to minimise cost) or a “Runaway Cost” (reasoning in infinite loops to maximise accuracy) because these objectives are orthogonal. Importantly, a probabilistic model functioning within the very state-space it is exploring is unable to objectively assess its own halting condition, the Value of Computation (VoC). Thus, it is theoretically necessary to have an external, deterministic meta-control layer.

1.3 The Proposed Solution: The System-3/0 Blueprint and MAS-OS

To address these systemic vulnerabilities, we introduce the System-3/0 Blueprint, a domain-agnostic orchestration framework, and its application layer, the Multi-Agentic System Operating System (MAS-OS). The core design principle is the strict separation of orchestration from execution, enforced by typed contracts.

This separation is operationalised by the architecture through different layers:

System-3 (The Kernel Scheduler)

It acts as the meta-controller. System-3 determines what runs next, not the LLM. It uses a mathematically optimal VoC halting rule to enforce global token budgets (Ω) and dynamically synthesises execution Directed Acyclic Graphs (DAGs).

System-0 (The Protection Ring)

It operates as the system-call boundary. It functions as a deterministic safety envelope by using constrained decoding to strictly project each agent output onto a valid JSON Schema before it reaches downstream agents.

Systems 1 and 2 (The Execution Processes)

System-1 comprises Small Language Model (SLM) specialists with *exclusive* tool access: file operations, code execution, web search, and persistent memory. System-2 comprises LLM-powered deliberate reasoners that hold *no tools directly* and access the environment solely by spawning System-1 sub-agents. This architectural constraint ensures that every tool interaction, even those occurring mid-reasoning within a ReAct loop, passes through System-0 validation. As a result, the depth-k composability guarantee (Theorem 3.2) applies not only across top-level DAG steps but within reasoning chains themselves.

1.4 Primary Contributions

By bridging the gap between formal systems engineering and probabilistic generative modelling, we make the following theoretical and practical contributions:

1. **The Dynamic Agent Registry:** The multi-agent registry ($\mathcal{R}$) is formalised as a dynamic mapping from an `AgentID` to an `AgentSpec` 6-tuple, integrating real-time cost profiles (C_i) and capability tags (T_i).

2. **Safety Convergence and Depth-k Composability (Theorems 3.1, 3.2, and 3.3):** A mathematical proof demonstrating that System-0's projection layer reduces the probability of generating a syntactically invalid string to exactly zero, thereby guaranteeing that cascading syntactic failures are impossible at any recursion depth. Theorem 3.3 further establishes that unconstrained generation reliability vanishes exponentially with schema complexity, proving that System-0 is a topological necessity rather than a convenience.

3. **Optimal Agent Routing and Conformal Escalation (Theorems 3.4, 3.5, and 3.6):** The application of Weitzman's "Pandora's Box" problem to prove that System-3's Value of Computation gate is mathematically optimal, alongside a proof that the contextual routing policy achieves sublinear cumulative regret. Theorem 3.4 provides distribution-free escalation guarantees from System-1 to System-2 via conformal prediction.

4. **Architectural Rationale (Design Principle 3.1, Design Argument 3.2):** We articulate the information-theoretic motivation for the $\mathcal{O}_{\text{ReAct}}$ operator (Design Principle 3.1: tasks whose information demand exceeds the model's internal knowledge require external retrieval) and the engineering rationale for separating control from execution (Design Argument 3.2: scalarised reward functions produce sensitive policies, motivating an external meta-controller). These are presented as design arguments rather than formal theorems: the underlying quantities (mutual information, expected gain) are conceptual rather than computable, and the arguments motivate architectural choices without claiming impossibility for alternative designs.

5. **The Agentic OS Design:** A comprehensive system specification that treats goals as units of computation, establishes memory subsystems, and provides typed inter-process communication (IPC) for autonomous AI agents.

1.5 Scope and Theoretical Boundaries

The statements presented and the claims made in this paper are strictly scoped to syntactic guarantees in order to maintain intellectual rigour. Schema conformity, type safety, and budget adherence are all mathematically guaranteed by the framework. Since evaluating factual truth in generative models is still an empirical rather than a formally provable endeavor, we specifically do not assert absolute semantic correctness (e.g. , factual validity or total absence of hallucination). This approach guarantees that semantic errors are confined, auditable, and structurally contained by guaranteeing total syntactic safety.

To make this boundary precise: a structurally valid agent output (a well-formed JSON object with all required fields, correct types, and values within specified ranges) can still contain factually incorrect content. If an agent reports `{"answer": "Paris", "confidence": 0.95}`, System-0 guarantees that this object is parseable, that `confidence` is a float in $[0, 1]$, and that all downstream agents can consume it without structural errors. It does *not* guarantee that "Paris" is the correct answer. The contribution is containment: errors remain auditable single-agent failures rather than silent multi-agent corruptions.

It is critical to distinguish between the two classes of guarantees we provide. *Syntactic guarantees* (JSON Schema conformance, type safety, structural validity, and budget adherence) are mathematically provable and enforced deterministically by System-0. Every theorem in Section 3 operates strictly within this syntactic domain. *Semantic guarantees* (factual accuracy, logical correctness, and absence of hallucination) remain empirical properties of the underlying language models and are explicitly outside the formal claims of this work. The contribution of System-0 is not that it makes agents *correct*, but that it makes agent errors structurally contained, auditable, and unable to propagate syntactic corruption to downstream agents. This containment is what enables reliable composition of unreliable components, the central engineering insight of the System-3/0 Blueprint.

2 Related Work

This section reviews prior work that is closest to the System-3/0 Blueprint: model routing, multi-agent orchestration, constrained decoding, and contract-based verification. The goal is to identify exactly what existing frameworks solve and where the systems gaps still remain.

2.1 The Economics of Inference: LLM Routing and Cascading

The financial unsustainability of using frontier LLMs for all queries has driven significant research into dynamic model routing. Cascaded evaluation was first introduced by [CZZ24] on FrugalGPT, demonstrating that up to 98% cost reduction can be achieved by escalating queries to larger models only when smaller models fail. Building on this, [Din+24] created Hybrid LLM, which reduces dependency on massive parameter models by 40% by routing queries using trained quality-gap predictors. More recently, routing mechanisms have incorporated human preference data and uncertainty metrics. In order to maintain 95% of GPT-4's quality at a significantly lower cost, [Ong+25] introduced RouteLLM, which trains routers on 80,000 preference comparisons. [Su+25] provided a statistical method for uncertainty-aware model selection by applying conformal prediction to LLM routing via CP-Router. The closest previous theoretical work in this field is that of [Dek+25], who developed provably optimal algorithms for unified routing and cascading. Concurrently, Router-R1 applies reinforcement learning to train routing policies directly, and GraphRouter models routing as a graph classification problem over query embeddings.

At the protocol level, the Model Context Protocol (MCP) [Ant24] and Google's Agent-to-Agent (A2A) protocol [Goo25] have emerged as standardised interfaces for tool invocation and inter-agent communication respectively, though neither provides the formal output contracts or budget-aware routing that the System-3/0 Blueprint enforces.

The Gap: While these frameworks successfully optimize single-query, single-step execution, they do not address multi-step budget dynamics across execution Di-

rected Acyclic Graphs (DAGs), nor do they account for the composability guarantees required when routing outputs between disparate agents.

2.2 Optimal Search Theory and Contextual Bandits

To formalize when an agent should stop reasoning, we must look beyond NLP literature to classical economics and bandit theory. The "Pandora's Box" problem was established by [Wei78], who demonstrated that using a reservation-value ordering is the best method for searching alternatives with known costs. In the context of online learning and routing, [APS11] established the Contextual Upper Confidence Bound (LinUCB) algorithm proving sublinear cumulative regret for contextual bandits.

The Gap: These exacting mathematical bounds are fundamental to economics and reinforcement learning, yet they are rarely used in LLM agent routing. Current MAS rely on arbitrary token limits or heuristic step counts. By mapping Weitzman's reservation values to the System-3 agent selection process (Theorem 3.5) and applying LinUCB to achieve sublinear regret in dynamic agent registries (Theorem 3.6), we bridge that gap.

2.3 Multi-Agent Orchestration: Autonomy Without Guarantees

To handle tasks exceeding single-prompt capability, the field has widely adopted Multi-Agent Systems. AutoGen [Wu+24] pioneered conversational multi-agent patterns, allowing LLMs to coordinate on coding and logic tasks. Similarly, MetaGPT [Hon+24] introduced standard operating procedures (SOPs) for agent collaboration, while CrewAI enabled rapid role-based prototyping. LangGraph provides graph-based execution with production features such as checkpointing and streaming. DSPy [Kha+24] further abstracts prompt engineering into a declarative programming style, allowing models to compile optimized tool-calls.

The Gap: These frameworks lack rigorous output verification and cost-control mechanisms, despite their popularity. Instead of using typed inter-process communication, they rely on implicit string-passing protocols. As a result, they have weak composability guarantees; the pipeline may be silently corrupted by a malformed string generated at an early stage.

2.4 Syntactic Safeguards: Constrained Decoding

To mitigate the inherent unpredictability of generative models, recent advancements have focused on constrained decoding, forcing the model's token distribution to adhere to specific context-free grammars (CFGs). Willard and Louf [WL23] established the foundation with the Outlines framework, enabling efficient guided generation. Park et al. [Par+24] formalized Grammar-Aligned Decoding, proving bounds on distribution distortion when enforcing schemas. XGrammar, developed by Dong et al. [Don+24] in late 2024, scaled CFG enforcement to production workloads with nearly zero latency overhead. In a similar vein, the DOMINO engine [BFV24] showed quick, low-overhead constrained decoding that ensures outputs during generation are schema-valid.

The Gap: While constrained decoding solves the problem of syntactic validity at the individual generation level, it has not been formally integrated as a topological boundary (a "protection ring") across an entire multi-agent execution graph to guarantee depth-k composability.

2.5 Formal Verification and Agent Contracts

The unreliability of MAS has led to a recent push toward formal verification. Bhardwaj [Bha26] introduced Agent Behavioral Contracts (ABC), defining preconditions, invariants, and recovery mechanisms via Lyapunov drift analysis. Concurrently, the Agent Contracts framework (COINE 2026) [YT26] formalized resource-bounded contracts with hierarchical sub-contracting. Dewes and Dimitrova [DD25] explored contract-based design and verification for MAS, and Mikulski et al. [Mik+25] advanced assume-guarantee verification in stochastic multi-agent environments.

The Gap: These approaches predominantly rely on runtime behavioral monitoring, checking agent behavior probabilistically after the fact. They lack the deterministic, generation-time syntactic enforcement required to mathematically guarantee that invalid states cannot be generated in the first place.

2.6 The Efficacy Debate and the Resource Paradox

Empirical research has started to question the baseline utility of multi-agent systems as they become more complicated. Errors in MAS can cascade up to 17 times

in certain architectures, according to Kim et al.'s [Kim+25] evaluation of 180 agent setups. More importantly, when examined strictly under matched thinking-token budgets, Ekbote et al. [Ekb+26] demonstrated that single-agent LLMs often outperform multi-agent systems on multi-hop reasoning.

The Gap: This growing skepticism isolates the core problem we address. If unconstrained MAS architectures waste tokens and compound errors, the solution is not to revert to monolithic models, which suffer from Laziness Bias and Runaway Cost, but to architect a formal meta-control layer. The literature clearly indicates the necessity of marrying optimal stopping routing (System-3) with rigorous syntactic contract enforcement (System-0).

2.7 The Agentic Operating System (OS) Abstraction

The conceptualization of LLM systems as operating systems is an emerging frontier. Packer et al. [Pac+23] introduced MemGPT, treating LLM context windows as RAM and external storage as disk memory, allowing agents to page context in and out. Mei et al. [Mei+24] proposed AIOS, an LLM agent operating system that abstracts resource allocation and context management. Tool learning frameworks like Gorilla [Pat+23] and Toolformer [Sch+23] established the precedents for isolated "system calls" by teaching models to invoke APIs reliably. Furthermore, benchmarking suites like OSWorld [Xie+24] and WebArena [Zho+23] have emerged to assess these systems in realistic computing environments.

The Gap: While AIOS and MemGPT successfully abstract memory and context, they lack the formal execution split proposed by the System-3/0 Blueprint. The MAS-OS we present treats the goal as the unit of computation, System-3 as the kernel scheduler, and System-0 as the system-call boundary. By routing all File, Python, and Web interactions through strict JSON-Schema-validated contracts, MAS-OS provides the process isolation, typed IPC, and deterministic safety guarantees required for production-scale autonomy.

3 Mathematical and Formal Framework

3.1 Global Problem Formulation

This section states the formal model used in this paper. The System-3 orchestrator is modeled as a policy optimizer under explicit compute constraints, not as a heuristic classifier.

Definition 3.1 (The Agentic Utility Function). *Let $\mathcal{Q}$ be the space of all possible user queries. For a given query $q \in \mathcal{Q}$, we possess a portfolio of reasoning models $\mathcal{M} = \{m_{slm}, m_{llm}\}$. We define a Utility Function $U(m, q)$ representing the quality of the answer and a Cost Function $C(m)$ representing computational expense. The objective is to find a routing policy $\pi : \mathcal{Q} \to \mathcal{M}$ that maximizes expected utility:*

$$\max_{\pi} \mathcal{J}(\pi) = \mathbb{E}_{q\sim\mathcal{D}}\big[U(\pi(q), q) - \lambda C(\pi(q))\big] \tag{3.1}$$

Subject to the global budget constraint:

$$\sum_{t=0}^{T} C(\pi(q_t)) \leq \Omega \tag{3.2}$$

Where λ is a Lagrange multiplier controlling the user's tolerance for cost versus accuracy.

Lemma 3.1 (The Efficiency Condition). *This lemma shows when hierarchical routing (System-3) is strictly better than always calling a monolithic LLM.*
Statement: Let $\Delta C = C(m_{llm}) - C(m_{slm})$ be the marginal cost saving of the SLM. Let $\epsilon(q)$ be the Confidence Gap. The System-3 architecture yields a strictly positive utility gain ($\Delta\mathcal{J} > 0$) if and only if the Joint Efficiency Ratio (JER) satisfies:

$$\frac{\mathbb{E}[\Delta C]}{\mathbb{E}[\epsilon(q)]} > \frac{\alpha}{\lambda} \tag{3.3}$$

Proof sketch: Decompose $\mathcal{Q}$ into easy queries (where the SLM produces correct outputs) and hard queries (where escalation to the LLM is required). The utility differential between hierarchical and monolithic routing reduces to comparing the

cost saved on easy queries against the utility lost when the SLM is mistakenly used on hard queries; the JER inequality emerges as the threshold at which the former exceeds the latter. The full algebraic derivation follows the cascading-cost analysis of Chen et al. (2024). ■

3.2 System-0: The Deterministic Safety Envelope

Standard agents rely on "prompt engineering," which leaves a non-zero probability of failure. We model System-0 as a Projection Operator that reduces the entropy of the output space to zero for invalid states.

Definition 3.2 (The Projection Layer). *Let the Language Model define a probability distribution $P_\theta(y \mid x)$ over a vocabulary $\mathcal{V}$. Let $\mathcal{G}$ be a Context-Free Grammar (CFG) (e.g., JSON Schema) defining the valid language $L(\mathcal{G})$. System-0 acts as a filter that transforms P_θ into a constrained distribution Q_ϕ:*

$$Q_\phi(w_t \mid w_{<t}) = \frac{P_\theta(w_t \mid w_{<t}) \cdot \mathbb{I}(w_t \in S^t_{valid})}{Z_t} \tag{3.4}$$

Theorem 3.1 (The Constrained Decoding Safety Convergence).
Statement: For any strictly positive language model distribution P_θ, the probability of generating a syntactically invalid string $y \notin L(\mathcal{G})$ under the System-0 projection is exactly zero.
Proof: The probability of an error E (generating $w_{err} \notin S^t_{valid}$) is:

$$P(E) = \sum_{w \notin S^t_{valid}} Q_\phi(w) = \sum \frac{P_\theta(w) \cdot 0}{Z_t} = 0 \tag{3.5}$$

Thus, $P(E) \equiv 0$. ■

Theorem 3.2 (Depth-k Composability).
Statement: For an execution tree of depth k where System-0 enforces contracts at every node, and each agent S_i satisfies its contract with probability $(1 - \epsilon_i)$, the composed system satisfies the global specification with probability:

$$P_{composed} \geq \prod_{i=1}^{n} (1 - \epsilon_i) \geq 1 - \sum_{i=1}^{n} \epsilon_i \tag{3.6}$$

Proof: By structural induction on k. Because System-0 utilizes constrained decoding (Theorem 3.1), syntactic violations have $\epsilon_i = 0$. Therefore, $P_{composed} = 1$. Cascading syntactic failures are mathematically impossible. ■

Theorem 3.3 (Vanishing Probability of Validity).
Statement: As the complexity of the output schema $\mathcal{G}$ increases, the probability of a standard probabilistic model P_θ generating a valid string by chance converges to zero exponentially.
Proof: Let ϵ be the per-token error rate. The probability of generating a valid sequence of length L is $P(Valid) = (1-\epsilon)^L$.

$$\lim_{L\to\infty} (1-\epsilon)^L = 0 \tag{3.7}$$

System-0 acts as a projection operator Π that ensures $\epsilon = 0$ for syntactic violations. Without System-0, reliability tends to 0 as complexity grows; with System-0, syntactic reliability is 1 regardless of L. ■

3.3 The Sub-Agent Tuple and Dynamic Registry

We use calligraphic letters for sets: $\mathcal{A}$ for the agent space, $\mathcal{T}$ for the task space, and $\mathcal{M}$ for the model portfolio. Bold letters denote vectors. The quality function $Q(a, x) \in [0, 1]$ measures how well agent a handles task x. The cost function $C(a) > 0$ measures computational expense in token units.

Definition 3.3 (Sub-Agent Tuple). *Every agent in the framework instantiates the 4-tuple:*

$$S_i = \langle M_i, \mathcal{G}_i, \mathcal{K}_i, \mathcal{V}_i \rangle \tag{3.8}$$

Where M_i is the model; $\mathcal{G}_i$ is the Syntactic Boundary (a JSON Schema defining the valid language $L(\mathcal{G}_i)$); $\mathcal{K}_i$ is the Ephemeral Context; and $\mathcal{V}_i$ is the Verifier Function $\mathcal{V}_i : Output(M_i) \to \{0, 1\}$.

Definition 3.4 (Agent Registry). *The Agent Registry $\mathcal{R}$ is the dynamic mapping $\mathcal{R} : \texttt{AgentID} \to \texttt{AgentSpec}$, where $\texttt{AgentSpec}$ extends the 4-tuple to a 6-tuple:*

$$\texttt{AgentSpec} = \langle M_i, \mathcal{G}_i, \mathcal{K}_i, \mathcal{V}_i, C_i, T_i \rangle \tag{3.9}$$

- C_i: Cost Profile: a tuple of expected tokens, latency, and cost per token, updated dynamically from execution telemetry via exponential moving average.
- T_i: Capability Tags: a finite set of semantic labels (e.g. , `FILE_READ`, `PYTHON_EXEC`) declaring handled task types.

3.4 The Four Escalation Predicates

Definition 3.5 (Escalation Predicates). *Let S_i be executing task x with output o_t. Define the escalation condition ESCALATE$(S_i, x) := P_1 \vee P_2 \vee P_3 \vee P_4$ where:*

- P_1 **(Confidence Failure):** $\text{conf}(o_t) < \tau_{sys1}$. Routes to Self-Consistency.
- P_2 **(Schema Violation):** $\mathcal{V}_0(o_t) = 0 \wedge \text{repair}(o_t) = \text{NULL}$. Routes to Verifier-Retry.
- P_3 **(Capability Declaration):** S_i explicitly signals `CAPABILITY_EXCEEDED`. Routes to Decomposition.
- P_4 **(Context Overflow):** $|context(x)| > context\_limit(M_i)$. Routes to Decomposition.

Corollary 3.1 (Composability of Agents). *Two agents S_A and S_B are Safely Composable if and only if the output language of A is a subset of the input language of B:*

$$L(\mathcal{G}_{out}^{A}) \subseteq L(\mathcal{G}_{in}^{B}) \tag{3.10}$$

3.5 System-1 to System-2: Conformal Escalation

Instead of hand-tuned confidence thresholds, we use conformal prediction to define a principled escalation condition from System-1 to System-2.

Definition 3.6 (Nonconformity Measure). *Let $s(x, y) \in \mathbb{R}$ be a nonconformity scoring function that measures how unusual a proposed response y is for a query x. Higher scores indicate lower confidence.*

Theorem 3.4 (Conformal Coverage Guarantee).
Statement: Given a user-defined error tolerance $\alpha \in (0, 1)$ and assuming exchangeability of calibration and test data, System-1 produces a prediction set $\mathcal{C}_\alpha(x)$ that contains the ground-truth valid response with probability at least $1 - \alpha$, independent of the underlying data distribution.
Proof: Assume a calibration dataset $\mathcal{D}_{cal} = \{(x_i, y_i)\}_{i=1}^{n}$. Assuming data exchange-

ability, we compute calibration scores $s_i = s(x_i, y_i)$. For a new test query x_{test}, we define the prediction set as:

$$\mathcal{C}_\alpha(x_{test}) = \{y \in \mathcal{Y} : s(x_{test}, y) \leq \hat{q}\} \tag{3.11}$$

Where $\hat{q}$ is the $\frac{\lceil (n+1)(1-\alpha) \rceil}{n}$ empirical quantile of the calibration scores $\{s_1, \ldots, s_n\}$. By the fundamental theorem of conformal prediction (Vovk et al., 2005), because the true test score s_{test} is exchangeable with the calibration scores, the rank of s_{test} is uniformly distributed. Therefore:

$$P(y_{test} \in \mathcal{C}_\alpha(x_{test})) = P(s_{test} \leq \hat{q}) \geq 1 - \alpha \tag{3.12}$$

Escalation Corollary: If $|\mathcal{C}_\alpha(x_{test})| > 1$, System-1 cannot confidently yield a single deterministic action at the $1 - \alpha$ level. The System-3 orchestrator uses this strict mathematical bound as the trigger to escalate the query to System-2. ■

Practical instantiation of the nonconformity measure. We instantiate $s(x, y)$ as Jaccard disagreement between two independent System-1 generations of the same query: $s(x, y) = 1 - J(y_1, y_2)$. This is model-agnostic and does not require token log-probabilities, which are not consistently exposed by local inference backends. The trade-off is weaker finite-sample sharpness than logit-based scores: coverage remains marginal (not conditional), and exchangeability may degrade under distribution shift.

3.6 System-3 Routing: Optimal Stopping and Regret Bounds

Routing in System-3 is treated as a search problem over K candidate agents or tools, balancing expected quality and compute cost.

Definition 3.7 (Value of Computation). *At step t in a System-2 reasoning chain, the Value of Computation is:*

$$VoC(s_t) = \mathbb{E}[\Delta V(s_{t+1}) \mid s_t] - Cost(E_t) \tag{3.13}$$

The optimal stopping rule dictates halting when additional reasoning produces non-positive expected net value: $T^* = \min\{t : VoC(s_t) \leq 0\}$.

Theorem 3.5 (Optimal Agent Routing under Known Quality Distributions).
Statement: Assume the quality distributions $F_k(q)$ of all K candidate agents are

known a priori. Then the optimal policy for System-3 routing maps exactly to Weitzman's "Pandora's Box" problem: compute a static reservation value σ_k for each agent, query agents in descending order of reservation value, and halt when the observed quality exceeds the best remaining reservation value.

Proof: Let there be K available agents. Querying agent k costs $c_k > 0$ and returns a response of quality q_k drawn from a known cumulative distribution function $F_k(q)$. We define the reservation value σ_k as the solution to the equation:

$$c_k = \int_{\sigma_k}^{\infty} (q - \sigma_k) dF_k(q) \tag{3.14}$$

This equation equates the marginal cost of querying agent k (c_k) with the expected marginal benefit of finding a quality score strictly greater than σ_k. By Weitzman (1979), the optimal selection rule is to sort the agents such that $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_K$. The optimal stopping rule dictates that System-3 should halt the search and return the best observed response $\hat{q}$ as soon as:

$$\hat{q} \geq \max_{j \in unqueried} \sigma_j \tag{3.15}$$

This proves that System-3's Value of Computation (VoC) gate is mathematically optimal under bounded resources. ■

Assumption realism and finite-sample behavior. Theorem 3.5 assumes known quality distributions F_k; in deployment these are estimated online from telemetry via the registry cost profile. Hence, optimality is asymptotic and early routing quality is warm-up limited by estimation error in $\hat{F}_k$.

Theorem 3.6 (Sublinear Regret under Linear Reward Assumption).

Statement: A System-3 orchestrator deploying a Contextual Upper Confidence Bound (LinUCB) routing policy achieves sublinear cumulative regret, proving that the system mathematically converges on the optimal model allocation over time.

Proof: Under the linear reward assumption, LinUCB selects at each step:

$$k_t = \arg\max_k \left(\hat{\theta}_k^T x_t + \alpha \sqrt{x_t^T A_k^{-1} x_t} \right) \tag{3.16}$$

where $\alpha > 0$ is the exploration parameter and $A_k = I_d + \sum_\tau x_\tau x_\tau^T$ is the design matrix for agent k. Standard contextual bandit theory (Abbasi-Yadkori et al., 2011) establishes that the cumulative regret $R(T)$ is bounded by:

$$R(T) \leq \mathcal{O}\left(d\sqrt{KT \log\left(\frac{KT}{\delta}\right)} \right) \tag{3.17}$$

with probability $1 - \delta$. Since $\lim_{T\to\infty} R(T)/T = 0$, the System-3 router converges to the optimal cost-accuracy routing frontier. ■

Assumption realism and feature representation. The linear reward model $\mathbb{E}[U_{k,t} \mid x_t] = \theta_k^T x_t$ is an approximation; we use a deterministic 64-dimensional hash projection for model-agnostic features. The regret guarantee remains asymptotic, while finite-sample performance depends on feature quality and reward volume during warm-up.

3.7 Formal Dynamics of Cognitive Operators (System-2 Expansion)

Definition 3.8 (Operator Space). *We formally distinguish three classes of System-2 cognitive operators:*

- $\mathcal{O}_{CoT}$ **(Chain of Thought):** An internal autoregressive transition. $z_{t+1} \leftarrow P_\theta(z_{t+1} \mid z_t)$
- $\mathcal{O}_{ReAct}$ **(Reasoning + Acting):** A coupled transition involving an environmental oracle. $z_{t+1} \leftarrow P_\theta(z_{t+1} \mid z_t, obs_t)$ where $obs_t \sim \mathcal{E}(action_t)$
- $\mathcal{O}_{ToT}$ **(Tree of Thoughts):** A branching transition using Monte Carlo Search.

Design Principle 3.1: Tool Requirement for Knowledge-Intensive Tasks

Principle. A task T cannot be solved by pure internal reasoning ($\mathcal{O}_{\text{CoT}}$) when the information required to solve it exceeds what is encoded in the model weights. The agent must invoke an external information source ($\mathcal{O}_{\text{ReAct}}$) when the task's information demand cannot be met from internal knowledge alone.

Information-theoretic framing. Let $I(\theta; T)$ denote the (conceptual) mutual information between the model weights θ and the task domain $\mathcal{D}_T$, and let $H(T \mid x)$ denote the conditional entropy of the solution given the input. The principle can be stated as: external retrieval is required when

$$H(T \mid x) > I(\theta; T). \tag{3.18}$$

The quantities $I(\theta; T)$ and $H(T \mid x)$ are not directly measurable in current language model architectures; we use them as conceptual labels to make the principle precise rather than as values to be computed.

Architectural consequence. In $\mathcal{O}_{\text{CoT}}$, the entropy reduction at each reasoning step is bounded by what is encoded in θ. For tasks where the required information lies outside the model's training distribution, no amount of internal reasoning closes the gap. $\mathcal{O}_{\text{ReAct}}$ introduces an external entropy source $\mathcal{E}$ via tool invocation, providing the missing information channel. This motivates the architectural rule, enforced by the System-3 routing policy, that tasks involving real-world facts or up-to-date data are routed to agents with retrieval tools.

3.8 Budget-Aware Evaluation Metrics

To explicitly penalize token waste and measure the success of the System-3/0 architecture, we define four budget-aware metrics:

Schema Compliance Rate (V_s) Measures the deterministic reliability of System-0.

$$V_s = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}(y_i \in L(\mathcal{G})) \tag{3.19}$$

Budget Efficiency Score (E_b) Quantifies intelligence per compute unit.

$$E_b = \frac{\sum \mathbb{I}(Success_i)}{\sum Tokens_i} \times 1000 \tag{3.20}$$

Safety Compliance Score (C_{safe}) A binary-weighted penalty metric for harmful tool calls.

$$C_{safe} = 1 - \frac{\text{Critical\_Failures}}{N} \tag{3.21}$$

Joint Performance Index (JPI) A unified objective function combining accuracy, structure, and cost.

$$JPI = w_1 \cdot Accuracy + w_2 \cdot V_s + w_3 \cdot \log(E_b) \tag{3.22}$$

Design Principle 3.1 articulated why pure chain-of-thought reasoning ($\mathcal{O}_{\text{CoT}}$) is insufficient for tasks whose information complexity exceeds the model's stored knowledge, motivating the System-2 operator hierarchy that includes CoT, ReAct, and Tree of Thoughts. We now turn to the complementary question: why a single policy controlling these operators cannot reliably self-regulate, motivating the separation of control from execution that defines the System-3 layer.

3.9 Architectural Rationale: Why a Separated Meta-Controller

Design Argument 3.2: Motivation for an External Meta-Controller

Argument. A single policy π cannot robustly balance solution accuracy against resource consumption when both objectives are scalarised into a single reward function. Extreme settings of the trade-off parameter produce pathological policies: aggressive cost penalties yield "Laziness Bias" (premature termination), while permissive cost penalties yield "Runaway Cost" (non-terminating reasoning loops). This sensitivity motivates separating the stopping decision from the reasoning policy.

Discussion. Consider a monolithic System-2 policy π_{mono} optimising a scalarised joint reward

$$\mathcal{R} = \mathcal{R}_{\text{accuracy}} - \lambda \mathcal{R}_{\text{cost}}. \tag{3.23}$$

Two regimes illustrate the difficulty:

- **High λ (cost-sensitive).** The optimal action becomes immediate termination with a low-confidence response, since $\mathcal{R}_{\text{cost}} \approx 0$ dominates the objective.

- **Low λ (accuracy-sensitive).** The model has no incentive to terminate and may extend reasoning indefinitely, since the marginal cost is negligible relative to potential accuracy gains.

The Value of Computation $\text{VoC}_t = \mathbb{E}[\Delta\mathcal{R}_{\text{accuracy},t+1}] - \text{Cost}$ is the natural quantity for deciding when to halt. However, estimating $\mathbb{E}[\Delta\mathcal{R}_{\text{accuracy}}]$ requires the model to evaluate the expected gain from continuing to reason, a meta-level judgment about the reasoning chain itself. A policy operating inside its own state-space cannot reliably evaluate its own halting condition.

Architectural consequence. An external controller (System-3) that observes the reasoning trace from outside can incorporate auxiliary signals (global token budget, predicate firings from System-0, the LinUCB-derived reservation values from §3.6) that are invisible to the reasoning policy. We adopt this separation as a robustness strategy. This is not a proven impossibility result for monolithic policies; it is an engineering argument that separating control from execution produces more predictable behaviour under real-world budget constraints.

4 Architectural Framework

4.1 Design Principles

This section specifies how the System-3/0 architecture is organized and why each boundary exists. Three design principles drive the implementation:

1. Every component is a registered agent with a formal specification (the `AgentSpec`), not a hardcoded function.

2. All inter-agent communication is governed by System-0 schema validation enforced at the output generation level, avoiding reliance on implicit string-passing conventions.

3. The global budget Ω is encoded strictly in System-3's routing logic through the Value of Computation (VoC) criterion, rather than imposed as a post-hoc limit that can be bypassed.

4.2 System-3: The Meta-Controller

System-3 is the orchestrator. It does not route over a hardcoded agent list; it routes over the runtime Agent Registry $\mathcal{R}$.

Upon receiving a task, System-3 performs four sequential operations:

1. **Classification:** Classifies task complexity using the **ClassifierAgent** as a lightweight pre-flight check.

2. **Discovery:** Queries $\mathcal{R}$.find() for capable agents, sorting them by their expected cost profile (C_i).

3. **Planning:** Builds an `ExecutionPlan` formulated as an ordered Directed Acyclic Graph (DAG) of `PlanSteps`, complete with budget allocations and topological dependencies.

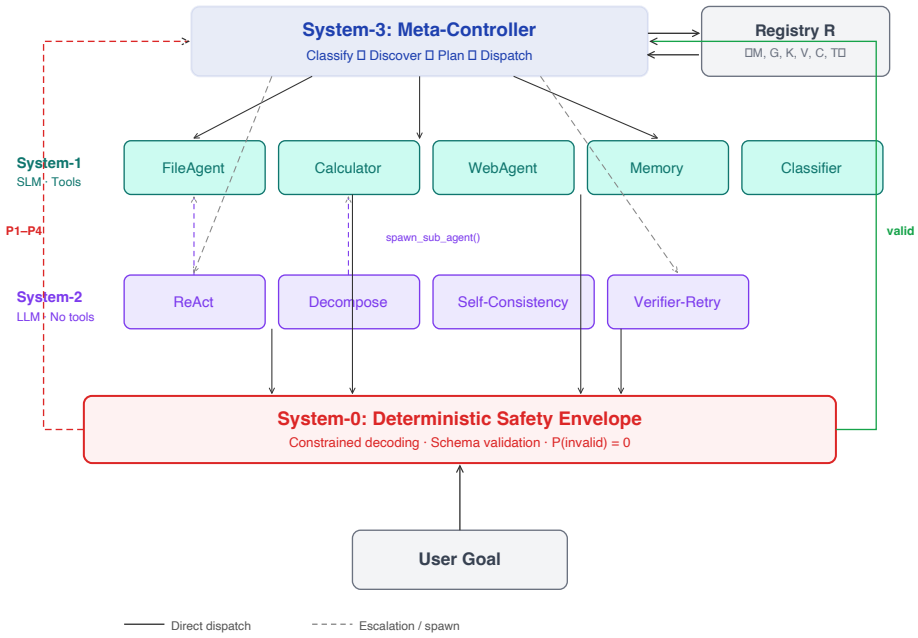

Fig. 4.1: The System-3/0 Blueprint architecture. System-3 routes tasks from the Agent Registry. System-1 agents hold exclusive tool access. System-2 agents access the environment only by spawning System-1 sub-agents. System-0 validates every output at every boundary.

4. **Dispatch & Orchestration:** Dispatches steps in dependency order, actively handling escalations by selecting appropriate System-2 agents based on the specific predicate triggered during execution.

4.3 System-1: Fast Heuristic Agents

System-1 contains fast specialist agents with **exclusive** access to external tools. These are SLM-backed workers with narrow responsibilities. No other tier directly invokes tools; System-2 reaches tools only through spawned System-1 sub-agents (Section 4.4).

FileAgent: Reads, writes, and lists real files on disk. *Tools:* `file_read`, `file_write`, `file_list`, `file_csv`. It serves as the OS file system interface.

CalculatorAgent: Runs isolated Python code for exact arithmetic. *Tools:* `python_exec`. This prevents numerical hallucination by executing real code and returning deterministic results.

WebAgent: Searches the live web via DuckDuckGo. *Tools:* `web_search`, `web_fetch`. Returns verifiable search results rather than confabulated information.

MemoryAgent: Persists and retrieves information across sessions. *Tools:* `memory_store`, `memory_recall`, `memory_list`. This lets agents recover context and state from prior executions.

ClassifierAgent: Estimates task complexity using a fast LLM call. It possesses no tools, but critically helps System-3 plan budgets before committing to an expensive execution path.

4.4 System-2: Deliberate Reasoning Agents

System-2 contains deliberate LLM reasoners for harder multi-step tasks. These agents hold **no direct tools**. When they need external interaction (file/code/web), they spawn System-1 workers. Returned outputs pass through System-0 before re-entering the reasoning loop. This is what makes Theorem 3.2 apply inside reasoning chains, not just across top-level DAG edges. We use five cognitive strategies:

ReAct Agent: Interleaves reasoning with tool interactions by spawning System-1 sub-agents for each action step. At each iteration, it produces a thought, identifies the required tool, dispatches a System-1 agent to execute it, receives the System-0-validated observation, and either continues reasoning or produces a final answer.

Self-Consistency Agent: Samples K independent answers using diverse templates, computes a Jaccard similarity vote, and sets confidence as the winning fraction.

Tree of Thoughts (ToT) Agent: Generates N independent reasoning branches using different approaches (analytical, intuitive, first-principles). It scores each branch using an LLM on a scale of 0–10, returning the highest-scoring branch.

Verifier-Retry Agent: Generates an initial answer and passes it to a separate LLM that acts as a critic. The critic checks for correctness and provides explicit error descriptions, prompting a retry with error context if the initial output was incorrect.

Decomposition Agent: Breaks the problem into sub-problems, spawning System-1 sub-agents to answer each in sequence with accumulated context, synthesizing

them into a final answer. Sub-agents implement a process-forking analogy: the parent allocates a budget slice to the child, and the child executes independently before returning the answer to the parent.

4.5 System-0: The Deterministic Protocol

System-0 is not another agent. It is the contract boundary that *every* execution path must cross, including internal System-2 tool calls executed via System-1. Uniform enforcement at this boundary is what makes the depth-k composability guarantee (Theorem 3.2) hold at arbitrary recursion depth.

1. **Inline Enforcement:** The runtime's `format='json'` flag constrains generation to JSON-parseable outputs. This implements the constrained decoding safety guarantee, $P(\text{invalid syntax}) = 0$. It is not merely convenient; Theorem 3.3 shows that unconstrained validity probability falls exponentially with schema complexity.

2. **Post-Hoc Schema Validation:** The generated text is parsed and validated against the agent's registered output schema using Draft-7 JSON Schema validation. Field-level repair is attempted for out-of-range values (e.g. , clamping confidence to $[0, 1]$) and missing optional fields (e.g. , substituting schema defaults).

3. **Escalation Predicate Evaluation:** Predicates P_1 through P_4 are evaluated in strict order on the validated output. The first triggered predicate is returned to System-3 as the escalation reason, driving the routing logic.

The System-0 audit log records every validation event with `agent_id`, `task_id`, `schema_name`, the status (`valid/repaired/failed`), the triggered predicate, and a timestamp. This enables post-hoc computation of metrics and provides the full traceability necessary for production deployments claiming formal guarantees.

4.6 Escalation Routing Protocol

When a System-1 agent fails to complete a task confidently and validly, System-0 flags the specific failure mode (Predicates $P_1 \dots P_4$). The System-3 orchestrator uses this signal to route the task to the optimal System-2 cognitive strategy. This mapping ensures that compute is only spent solving the exact modality of failure encountered.

Tab. 4.1: Escalation Routing Matrix: Mapping System-0 Predicates to System-2 Strategies

Predicate	Failure mode		System-3 resolution routing
P_1	**Confidence failure**	$\texttt{conf} < \tau_{\text{sys1}}$	**Self-Consistency Agent** Reduces epistemic uncertainty through a $K{=}3$ majority vote sampling.
P_2	**Schema violation**	$\mathcal{V}_0(o_t) = 0$	**Verifier-Retry Agent** Regenerates the response utilizing the exact schema violation feedback as context.
P_3	**Capability exceeded**	`CAPABILITY_EXCEEDED`	**Decomposition Agent** Breaks the parent task into discrete subtasks that fit within standard capability limits.
P_4	**Context overflow**	$\lvert\text{context}\rvert > \text{limit}$	**Decomposition Agent** Breaks the execution into subtasks to process massive contexts in parallel, isolated windows.

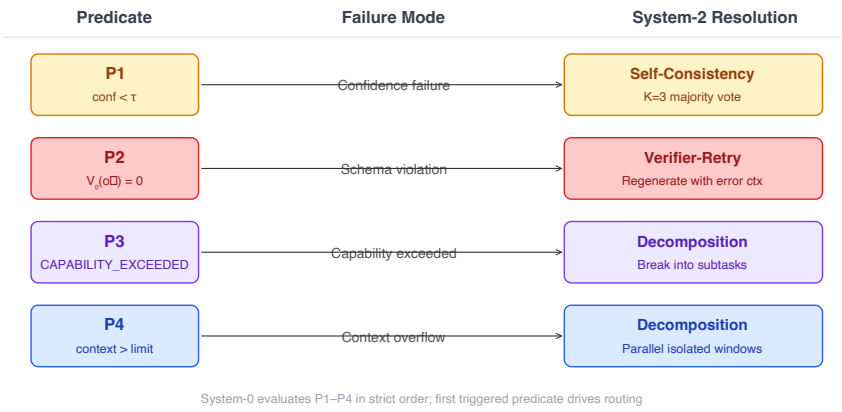

Fig. 4.2: Escalation routing flow. System-0 evaluates predicates P1–P4 on every agent output. The first triggered predicate determines the System-2 recovery strategy selected by System-3.

5 Implementation

This section describes the reference implementation of the System-3/0 Blueprint as a Python package. It covers the stack, model allocation, package layout, and reproducibility details. The goal is to show that the formal design can run on consumer hardware with local models.

5.1 Technology Stack

The framework is implemented as a Python package (`mas/`) with roughly 20 modules. Execution is built on LangGraph `StateGraph`: agents are nodes, routing is expressed through conditional edges, and a typed `MASState` dataclass acts as the message bus. The graph is constructed from the runtime registry, so the framework does not require a fixed compile-time agent set.

The full dependency stack consists of: LangGraph for graph execution and state management; LangChain Core for agent abstractions; Ollama for local model inference with no API keys required; `jsonschema` for System-0 Draft-7 JSON Schema validation; and `matplotlib` for evaluation visualisation. No external API dependencies are required at any point in the pipeline. The complete reference implementation is available at `https://github.com/Jyotirmoy17/System-3-0-MAS-OS`.

5.2 Model Assignment

System-1 uses `phi3:mini` (3.8B parameters, approximately 2.5 GB RAM). This model was selected for the fast heuristic tier because it provides low latency, low cost, and reliable schema-bound output for structured tasks. System-2 uses `llama3.1:8b` (8B parameters, approximately 5 GB RAM), selected for stronger multi-step reasoning capability.

Both models run locally via Ollama on consumer hardware. Relative cost units in the evaluation are set as SLM $= 1.0$ and LLM $= 8.0$. These units come from observed latency ratio on the reference machine and are used for Budget Efficiency Score (E_b) and Joint Performance Index (JPI).

5.3 Hardware and Reproducibility

All experiments run on an AMD Ryzen 5 7000 series machine with 24 GB RAM and no discrete GPU. `phi3:mini` loads at approximately 2.5 GB and `llama3.1:8b` at approximately 5 GB; both fit concurrently with headroom for Python. Hyperparameters, including τ_{sys1}, exploration parameter α, and budget allocation ratios, are centralized in one configuration object for reproducibility.

5.4 Package Structure

The implementation is organised as follows:

- `mas/core/registry.py`: The Agent Registry $\mathcal{R}$. Implements the 6-tuple `AgentSpec`, `CostProfile` with exponential moving average updates, and the `Capability` taxonomy. Supports `register()`, `find()`, `remove()`, and `update_cost()` operations (Section 5.4.1).

- `mas/core/base_agent.py`: Defines the `BaseAgent` abstract class and typed `TaskInput`/`AgentOutput` dataclasses. All agents inherit from this class. Includes `spawn_sub_agent()` for System-2 to System-1 delegation.

- `mas/core/system0.py`: System-0 contract enforcement. Implements constrained decoding via Ollama's `format='json'` flag, post-hoc Draft-7 JSON Schema validation, field-level repair, and the four escalation predicate evaluators (P1–P4). Maintains a full audit log.

- `mas/core/orchestrator.py`: System-3 orchestrator. Implements the VoC calculator, budget manager tracking the global token budget Ω, the routing decision engine with JER computation, and the LangGraph `StateGraph` builder. The graph is constructed dynamically from the registry.

- `mas/core/ollama_client.py`: Ollama REST API wrapper with JSON mode enforcement and token counting.

- `mas/tools/`: Five tool implementations: `FileTool` (sandboxed file I/O), `PythonTool` (isolated code execution with whitelisted imports), `WebTool` (DuckDuckGo search and HTML fetch), `MemoryTool` (persistent key-value store), and `ShellTool` (whitelisted commands).

- `mas/agents/s1/`: System-1 agent implementations: FileAgent, CalculatorAgent, WebAgent, MemoryAgent, and ClassifierAgent.

- `mas/agents/s2/`: System-2 agent implementations: ReActAgent, DecompositionAgent, SelfConsistencyAgent, TreeOfThoughtsAgent, and VerifierRetryAgent.

- `mas/agents/defaults.py`: Default registry with all 10 agents pre-registered.

- `mas/eval/`: The formal evaluation harness. Contains the execution pipelines (`harness.py`, `batch.py`), the budget-aware metric computation and statistics engine (`statistics.py`, `composite_metrics.py`), and the automated results generator (`results.py`).

5.4.1 Agent Registry API

The Agent Registry exposes four operations at runtime, each enforcing the invariant that every agent in the system is a fully specified 6-tuple:

- `R.register(spec: AgentSpec)`: Inserts a new agent into the registry. The registry validates that the `AgentSpec` contains a well-formed JSON Schema (G_i), a non-empty capability tag set (T_i), and a valid cost profile (C_i) before accepting the registration.

- `R.find(capabilities: Set[str], tier: int) -> AgentSpec`: Returns the cheapest registered agent whose capability tags are a superset of the requested set. This is the primary interface used by System-3 during the *Discovery* phase of planning.

- `R.remove(agent_id: str)`: Idempotent deletion. Removes the agent from the registry and prevents future routing to it. Active executions are not interrupted.

- `R.update_cost(agent_id: str, tokens: int, latency: float)`: Updates the agent's cost profile using an Exponential Moving Average (EMA) with a smoothing factor of $\alpha = 0.3$. This ensures that routing decisions reflect recent execution performance rather than stale initialisation values.

Because System-3 operates over $\mathcal{R}$ at runtime rather than a hardcoded agent set, adding a new domain capability, for example a `DatabaseAgent` for SQL queries, requires only a single `R.register()` call with the appropriate `AgentSpec`. No

framework code changes are needed, and the new agent automatically inherits all formal properties: System-0 contract enforcement, budget tracking, and escalation predicate evaluation.

5.5 System-0 Enforcement Pipeline

The System-0 enforcement pipeline operates in three stages on every agent output, implementing Theorem 3.1 in practice:

Stage 1 (Inline Enforcement): The Ollama client's `format='json'` flag constrains the token distribution during generation to produce only JSON-parseable outputs. This implements the constrained decoding projection Q_ϕ from Definition 3.2.

Stage 2 (Post-Hoc Validation): The generated JSON is validated against the agent's registered output schema using Draft-7 JSON Schema validation. Field-level repair is attempted for out-of-range values (e.g., clamping confidence to $[0, 1]$) and missing optional fields (e.g., substituting schema defaults).

Stage 3 (Predicate Evaluation): Predicates P1 through P4 are evaluated in strict order on the validated output. The first triggered predicate is returned to System-3 as the escalation reason.

The System-0 audit log records every validation event with `agent_id`, `task_id`, `schema_name`, status (`valid/repaired/failed`), the triggered predicate (if any), and a timestamp.

5.6 Confidence Estimation

System-1 confidence estimation uses a two-sample self-consistency method. For each query, the SLM generates two independent responses. The Jaccard similarity between the responses serves as a proxy for confidence: if both responses agree substantially (Jaccard $\geq \tau_{\text{sys1}}$), the agent is confident. If they diverge, predicate P1 fires and escalation to System-2 occurs.

The Jaccard agreement signal is the practical instantiation of the nonconformity measure $s(x, y)$ from Theorem 3.4: low inter-sample agreement maps to a high nonconformity score, triggering escalation in a manner that approximates the conformal prediction set $C_\alpha(x_{\text{test}})$ without requiring direct access to per-token logits.

This connection is what makes the System-1 escalation predicate principled rather than heuristic, even though the implementation cannot directly compute $\hat{q}$.

This approach is used because Ollama does not consistently expose per-token log-probabilities across all backends. Inter-sample agreement gives a model-agnostic confidence signal that works with local inference engines. The trade-off is granularity: direct logit access would provide a more precise confidence estimate.

6 Evaluation

6.1 Budget-Aware Metrics

Standard benchmarks often report accuracy without explicit cost controls. That is not enough for deployment settings where budget matters. This section uses five budget-aware metrics:

Schema Compliance Rate (V_s): Measures the deterministic reliability of System-0.

$$V_s = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}(y_i \in L(\mathcal{G})) \tag{6.1}$$

Budget Efficiency Score (E_b): Quantifies intelligence per compute unit.

$$E_b = \frac{\sum \mathbb{I}(Success_i)}{\sum Tokens_i} \times 1000 \tag{6.2}$$

Safety Compliance Score (C_{safe}): A binary-weighted penalty metric for harmful or out-of-bounds tool calls.

$$C_{safe} = 1 - \frac{N_{Critical\_Failures}}{N} \tag{6.3}$$

Cascading Failure Rate (CFR): The central empirical metric for validating Theorem 3.2. A cascading failure occurs when a syntactic violation at step i propagates uncaught to corrupt the output of step $j > i$.

$$CFR = \frac{N_{cascade}}{N_{total}} \tag{6.4}$$

Under the SYSTEM 0/3 condition, Theorem 3.2 predicts $CFR = 0$.

Joint Performance Index (JPI): A unified objective function combining accuracy, structural integrity, and cost.

$$JPI = w_1 \cdot Accuracy + w_2 \cdot V_s + w_3 \cdot \log(E_b) \tag{6.5}$$

JPI penalizes strategies that gain accuracy by overspending tokens, so we use it as the main summary metric in our evaluation.

6.2 Goal Suite

The evaluation suite contains three goal types, each chosen to stress a specific architectural property:

1. **Sequential Reasoning** (27 instances per condition). Decompose → Step 1 → Step 2 → Synthesise. Tests output-as-input chaining, the clearest cascade failure scenario, where a structural error at step 2 renders steps 3 and 4 incoherent.

2. **Code Analysis** (32 instances per condition). Parse → BugFind → Explain → Fix. Tests sub-agent spawning, where the Decomposition agent spawns the `FileAgent` and `WebAgent` for context. Key cascade scenario: a malformed bug list breaking the explainer module.

3. **Adversarial Reliability** (91 instances per condition). Deliberate schema violation injection at controlled pipeline steps. Directly tests System-0's P_2 catch mechanism, escalation to the Verifier-Retry agent, and successful recovery. This serves as the empirical proof of Theorem 3.2.

The total evaluation comprises 300 paired runs (150 per condition). All goals were written by the author manually, as disclosed in Section 8.2.

6.3 Experimental Conditions (Ablation Study)

To isolate the contribution of the System-3/0 architecture, the goal suite runs under two conditions (Table 6.1). The SYSTEM 0/3 condition runs the complete framework with System-0 contract enforcement and System-3 routing active. The BASELINE condition removes both System-0 contracts and System-3 routing, executing goals as a flat LangGraph pipeline without schema validation or optimal stopping, representing the default behaviour of current multi-agent frameworks.

Tab. 6.1: Experimental Conditions for Ablation Study

Condition	Description	System-0	Routing	What it tests
SYSTEM 0/3	Complete System-3/0	ON	ON	Full framework value (reference)
BASELINE	Contracts and routing removed	OFF	OFF	System-0 and System-3 necessity

6.4 Results

Table 6.2 presents the primary outcomes across both conditions. All statistical tests use the pre-registered significance threshold $\alpha = 0.05$.

Tab. 6.2: Performance Metrics across Experimental Conditions (300 runs, 150 per condition)

Condition	CFR	Accuracy	JPI	Mean Tokens	CR
SYSTEM 0/3	0.000	0.380	0.395	1,600	0.993
BASELINE	0.380	0.080	−0.131	5,724	1.000

CFR = Cascading Failure Rate. CR = Execution Completion Rate. Accuracy and JPI are exploratory (see Section 6.4.4). 95% CIs: $\text{CFR}_{\text{System 0/3}} \in [0.000, 0.025]$; $\text{CFR}_{\text{Baseline}} \in [0.306, 0.460]$.

6.4.1 Primary Result: Cascading Failure Rate

The central empirical result directly validates Theorem 3.2 (depth-k composability). Under the SYSTEM 0/3 condition, the Cascading Failure Rate is exactly 0.0% ($n = 150$, 95% CI $[0.000, 0.025]$). Under the BASELINE condition, the CFR rises to 38.0% (95% CI $[0.306, 0.460]$). A McNemar exact test confirms this difference is statistically significant ($b = 0$, $c = 57$, $p < 0.001$) with a large effect size (Cohen's $h = -1.328$, 95% CI $[-0.460, -0.300]$).

This result confirms the theoretical prediction: when System-0 enforces constrained decoding at every agent boundary, syntactic violations have $\epsilon_i = 0$, and the union bound yields $P_{\text{composed}} = 1$. When System-0 is removed, syntactic violations propagate freely across the execution graph, producing the observed 38% cascade rate.

Figure 6.1 visualises the CFR comparison. Figure 6.2 shows the per-step pipeline survival curve: the SYSTEM 0/3 condition maintains 100% structural validity at every pipeline depth (the blue line is constant at the 100% ceiling), while the BASELINE condition decays approximately as $(1 - \epsilon)^k$ with fitted $\epsilon = 0.092$, closely matching the theoretical prediction of Theorem 3.2.

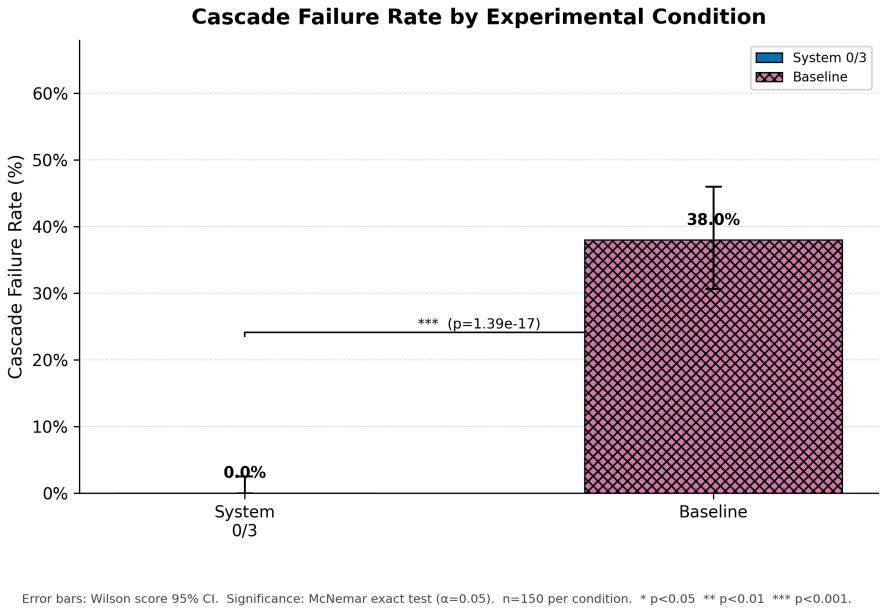

Fig. 6.1: Cascade Failure Rate by experimental condition. Error bars show Wilson score 95% CIs. Significance: McNemar exact test ($p < 0.001$). $n = 150$ per condition.

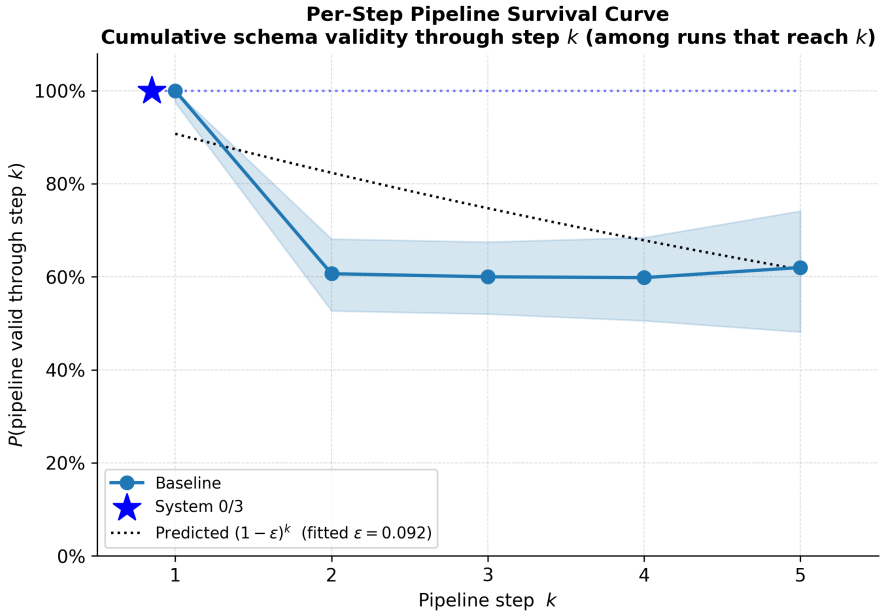

Fig. 6.2: Per-step pipeline survival curve. The System 0/3 line (blue) is constant at 100% across all pipeline depths, confirming Theorem 3.2. The Baseline decays approximately as $(1 - \epsilon)^k$ with fitted $\epsilon = 0.092$. Shaded region: 95% CI.

6.4.2 Cost Efficiency

The SYSTEM 0/3 condition uses a mean of 1,600 tokens per run (SD = 1,054), compared to 5,724 tokens for the BASELINE (SD = 2,754), a 3.6× reduction. A Wilcoxon signed-rank test confirms significance ($p < 0.001$, rank-biserial $\delta = -0.930$, 95% CI of the median difference $[-4{,}550, -3{,}701]$ tokens). This reduction arises from two mechanisms: System-3's routing avoids unnecessary LLM invocations for tasks the SLM can handle, and System-0's early catch of violations prevents wasted downstream computation on corrupted inputs.

6.4.3 Execution Completion

The BASELINE achieves a marginally higher execution completion rate (100.0%) compared to SYSTEM 0/3 (99.3%). This difference is not indicative of fragility in the framework. System-3's meta-controller performs intentional defensive early-stopping when adversarial data corruption is deemed unrecoverable, a design feature, not a failure mode. The Baseline achieves 100% completion by blindly emitting low-quality, corrupted single-step fragments without detecting corruption. True capability is reflected in the Accuracy and JPI metrics.

6.4.4 Accuracy and JPI

The SYSTEM 0/3 condition achieves an Accuracy of 0.380 (95% CI $[0.300, 0.460]$) compared to 0.080 for the Baseline (95% CI $[0.040, 0.127]$), and a JPI of 0.395 (95% CI $[0.349, 0.439]$) compared to -0.131 (95% CI $[-0.167, -0.095]$). The negative Baseline JPI reflects the logarithmic cost penalty in Equation 6.5: the Baseline expends 3.6× more tokens while achieving lower accuracy, producing a net-negative joint score.

7 The Agentic Operating System (MAS-OS)

This section describes MAS-OS, an OS-style application layer built on top of the System-3/0 kernel. It translates the formal framework into a practical runtime model for agent coordination.

7.1 Objective and Scope

MAS-OS treats a goal as a unit of computation, the System-3 Orchestrator as the kernel scheduler, and System-0 typed contracts as the system-call boundary. All agent outputs are validated by constrained decoding before reaching the next agent, ensuring malformed data cannot propagate.

In Scope: Kernel architecture; agent registry model; goal runtime and execution DAG; the tool interface as the protection ring; scheduler and event loop; inter-agent communication; memory subsystem; routing algorithms; and observability. The reference implementation runs entirely locally on a single 24GB RAM machine via Ollama, utilizing Phi-3 and Llama.

Out of Scope: Re-deriving the mathematical theorems (referenced from previous sections); production hardening concerns (rate limiting, multi-tenant billing, SSO); model training or fine-tuning procedures; and network-scale distribution (cross-datacenter sharding).

7.2 Background and the OS Analogy

LLMs can generate strong local reasoning traces, but production agent systems still need explicit runtime guarantees. MAS-OS adopts an operating-system analogy to organize those guarantees: scheduling, isolation, typed interfaces, persistent state, and failure handling.

Tab. 7.1: Mapping Traditional OS Concepts to MAS-OS

Traditional OS	MAS-OS	Implementation Note
Kernel	System-3 + System-0	Orchestration kernel with typed-contract syscall layer.
Process	Agent Instance	Registered `AgentSpec` with runtime state.
Process Table	Agent Registry ($\mathcal{R}$)	Dynamic mapping `AgentID` $\rightarrow \langle M, G, K, V, C, T \rangle$.
Scheduler	System-3 Planner	Weitzman reservation search + LinUCB + VoC gate.
Syscall Interface	Tool Layer	File, Python, Web, Memory, Shell (all schema-validated).
Memory Manager	Budget + MemoryTool	Global token budget Ω; persistent KV memory.
IPC	Typed AgentOutput	JSON-Schema-validated message passing across DAG nodes.
Fork / Spawn	`spawn_sub_agent()`	System-2 spawns System-1 for tool access. Depth-bounded; every spawn passes through System-0.
File System	`workspace/` tree	Sandboxed paths; all I/O strictly through FileTool.
Protection Ring	System-0 Envelope	Every output projected onto language $L(\mathcal{G})$ before propagation.

7.3 Design Goals and Non-Goals

Goals:

1. Syntactic violations between any two agents must be impossible at any recursion depth.

2. Every goal has a declared token budget Ω. Execution must terminate bounded by Ω.

3. Every routing decision is a solution to an optimization problem, not a hard-coded if/else tree.

4. Every error is a typed error; the system never silently returns invalid output.

5. New agents register their `AgentSpec` at runtime and inherit all formal properties automatically.

6. End users interact via a chat interface, while developers see a full live event telemetry stream.

Non-Goals: We do not claim universal *semantic* correctness of LLM outputs. Formal guarantees are intentionally restricted to syntax and type-level validity. Production multi-tenancy and model training pipelines are out of scope for Kernel v1.

7.4 Five-Layer Architecture

The MAS-OS application layer is organized into five tiers:

Layer 1: Kernel System-3/0 Blueprint. Handles routing, contracts, budget, and composability.

Layer 2: Tool Interface The syscall layer. Every real-world interaction is mediated and schema-validated here.

Layer 3: Agent Pool The living processes. System-1 specialists and System-2 reasoners residing in the dynamic Registry.

Layer 4: Goal Runtime Execution environment for goals, managing the Execution-Plan DAG, workspace state, and audit logs.

Layer 5: Interface The shell, including Web UI, CLI, REST APIs, and WebSocket event streams.

7.5 Detailed Component Design

7.5.1 The Kernel (Systems 0, 1, 2, and 3)

System-3 (The Control Plane): Scheduler, planner, budget manager, and router in a deterministic control module.
System-0 (The Envelope): System-call boundary and type checker implemented as a projection operator. System-3 chooses *what* to run; System-0 enforces output validity.
System-1 (Execution Workers): SLM-powered specialists (e.g., Phi-3). Fast, cheap, and narrowly capable. They hold the tools.
System-2 (Deliberate Reasoners): LLM-powered reasoners that hold *no tools directly* and access the environment exclusively by spawning System-1 sub-agents.

Every spawned tool call passes through System-0, ensuring the depth-k composability guarantee applies within reasoning chains.

7.6 Registry API

The registry supports four core operations at runtime:

- `R.register(spec)`: Inserts a new `AgentSpec`.
- `R.find(capabilities, tier)`: Returns the cheapest agent satisfying the required capability tags (T_i).
- `R.remove(id)`: Idempotent deletion of an `AgentSpec`.
- `R.update_cost(id, tokens, latency)`: Exponential Moving Average (EMA) update of an agent's cost profile (C_i) post-execution.

7.7 Goal and Execution Lifecycle

A goal is the unit of computation and contains five fields: description, domain, budget Ω, output schema, and trigger. System-3 translates this into an `ExecutionPlan` (a DAG of `AgentSteps`).

A user submits a goal via the interface. System-3 classifies its complexity, queries the registry, synthesizes the DAG, and allocates budgets. As steps dispatch, agents run tools and return an `AgentOutput`. System-0 gates every output. If an escalation predicate ($P_1 - P_4$) triggers, System-3 dynamically replans, routing to the appropriate System-2 recovery strategy.

7.8 The Tool Interface (The Protection Ring)

No agent touches the file system, network, or Python interpreter directly. All real-world access passes through a typed tool interface:

- **FileTool:** Sandboxed to the workspace. `file_csv` returns structured JSON, eliminating silent string-parsing errors.

- **PythonTool:** Isolated execution with whitelisted imports (math, statistics, json). Blocked from network/subprocess calls.

- **WebTool:** Structured DuckDuckGo search and HTML fetching.

- **MemoryTool:** Persistent key-value state across executions, enabling longitudinal agent memory through store and recall operations.

- **ShellTool:** Strictly whitelisted commands (`ls`, `grep`, `awk`) running inside the workspace.

8 Applications and Extensions

The System-3/0 Blueprint is domain-agnostic infrastructure. New applications are added by registering domain-specific agents; the core orchestration framework remains unchanged. In our evaluation, we mapped three distinct problem domains to the framework simply by registering specific agent configurations:

Code Analysis Application: Registers a CodeParserAgent and a BugFinderAgent (System-2/Decomposition). System-0 enforces structured bug reports (type, location, severity, fix), preventing free-text LLM hallucinations from breaking the downstream FixGeneratorAgent.

Sequential Reasoning Application: Executes a multi-hop logic pipeline (Decompose $\rightarrow$ Step 1 $\rightarrow$ Step 2 $\rightarrow$ Synthesise). This domain heavily exercises System-0's ability to maintain strict type safety across long chains, ensuring that outputs from Step 1 are perfectly formatted for ingestion by Step 2.

Adversarial Reliability Application: Injects deliberate data corruption into the environment. This domain exercises the System-3/0 escalation predicates, proving the orchestrator's ability to halt execution, detect unrecoverable schema violations, and route to a Verifier-Retry agent before downstream steps are corrupted.

8.1 Adversarial Reliability Demonstration

To empirically validate the depth-k composability theorem, we present a concrete adversarial trace. Consider a 4-step goal: "Research the impact of transformer architectures, identify key breakthroughs, explain each, and produce a structured summary."

Step 1 (System-1, WebAgent): Searches for transformer architecture papers. Returns a valid JSON object with `sources`, `snippets`, and `confidence` fields. System-0 validates: PASS. No predicate triggers.

Step 2 (System-1, FileAgent): Reads a local document for additional context. At this step, we deliberately inject a schema violation: the agent returns a malformed JSON object with a missing `content` field and `confidence` as a string instead of a float.

With System-0/3 (FULL_FRAMEWORK): System-0 catches the violation at Stage 2 (post-hoc validation). The missing `content` field cannot be repaired. Predicate P2 fires (`V_0(o_t) = 0`). System-3 receives the escalation signal and routes to the **Verifier-Retry Agent** (System-2). The Verifier-Retry agent regenerates the response with the exact schema violation as context, producing a valid output. Execution continues to Step 3 (analysis) and Step 4 (synthesis), both completing successfully. The goal succeeds.

Under the BASELINE Condition: The framework executes as a standard, flat pipeline without System-0 contracts or System-3 routing. The malformed output from Step 2 passes unchecked to Step 3. The analysis agent receives structurally invalid input; the missing `content` field causes it to reason over incomplete data, and the string-typed `confidence` value breaks any downstream numerical comparisons. Step 3 produces degraded output. Step 4 receives further degraded input and produces incoherent results. The goal fails with cascading errors at Steps 3 and 4.

Interpretation. This delta (0.0% cascade failure rate with System-0 versus 38.0% without ($p < 0.001$, $n = 150$ per condition)) is the central empirical result of the evaluation (Table 6.2), directly validating Theorem 3.2. Across 150 paired runs, 57 goals that cascaded under the Baseline were recovered under System 0/3 (McNemar $b = 0$, $c = 57$). The adversarial trace demonstrates that System-0's value is not merely theoretical: it converts a catastrophic multi-step pipeline failure into a recoverable single-step retry, while simultaneously reducing token expenditure by 3.6×.

8.2 Limitations and Future Work

The current version has clear limitations.

First, System-1 confidence estimation uses inter-sample agreement (Jaccard similarity between two independent SLM responses) as a proxy for true model confidence, because Ollama does not uniformly expose per-token log-probabilities. Direct logit access would provide a more fine-grained and theoretically grounded confidence signal.

Second, System-3's planner uses heuristic capability matching (set intersection on capability tags T_i) rather than a learned planner trained on execution traces. A learned planner could improve routing accuracy but would require a substantial training dataset of agent execution logs.

Third, the evaluation uses local models (`phi3:mini`, `llama3.1:8b`) rather than frontier API models (GPT-4, Claude). Results may differ in magnitude with stronger model pairs, though the framework's formal properties (constrained decoding safety in Theorem 3.1, depth-k composability in Theorem 3.2, and optimal routing in Theorem 3.5) are model-agnostic by construction.

Fourth, the evaluation tasks are author-designed rather than drawn from an established benchmark, as no existing multi-agent benchmark measures formal contract enforcement or cascading failure rates. The development of standardised benchmarks for these properties is an important direction for future work.

Fifth, the conformal coverage guarantee (Theorem 3.4) provides *marginal* coverage, not *conditional* coverage. Stronger results that guarantee coverage conditioned on specific query features would require conditional conformal prediction methods, which introduce additional computational overhead.

Finally, MAS-OS here is Kernel v1. Production concerns such as multi-tenancy, persistent agent lifecycles, distributed execution, and real-time streaming are out of scope in this version.

9 Conclusion

We presented the System-3/0 Blueprint, a domain-agnostic orchestration framework for Multi-Agent Systems. The framework targets three recurring deployment failures: cost drift, cascade fragility, and weak stopping control in long reasoning loops.

9.1 Summary of Contributions

The contributions span theory, architecture, and implementation.

At the theoretical level, the *safety theorems* proved that constrained decoding reduces the probability of syntactic violations to exactly zero (Theorem 3.1), that this guarantee composes to arbitrary recursion depth (Theorem 3.2), and that unconstrained generation reliability vanishes exponentially with schema complexity, establishing System-0 as a topological necessity (Theorem 3.3). The *routing theorems* established that System-3's routing via Weitzman reservation values is optimal under bounded resources (Theorem 3.5), that the contextual routing policy achieves sublinear cumulative regret (Theorem 3.6), and that conformal prediction provides distribution-free escalation guarantees from System-1 to System-2 (Theorem 3.4). The architectural rationale (Design Principle 3.1 and Design Argument 3.2) articulates why tasks requiring external information cannot be solved by pure chain-of-thought (motivating the $\mathcal{O}_{\text{ReAct}}$ operator), and why separating control from execution produces more robust behaviour under budget constraints than scalarised monolithic policies (motivating the System-3 meta-controller). These are stated as design arguments rather than formal theorems, in keeping with the conceptual rather than computable nature of the underlying quantities.

At the architectural level, the framework introduced the Dynamic Agent Registry as a runtime mapping from agent identifiers to formal 6-tuple specifications, enabling domain-agnostic extensibility. The design enforces that System-2 agents hold no tools directly and access the environment exclusively by spawning System-1 subagents, ensuring that Theorem 3.2 applies within reasoning chains, not only across top-level DAG steps. The four escalation predicates (P1–P4) provide a deterministic mapping from failure modes to recovery strategies, replacing ad-hoc error handling with principled routing.

At the implementation level, the reference implementation validates that these formal properties are realisable on consumer hardware (24 GB RAM, no GPU) using fully local, open-source models via Ollama, with zero external API dependencies. A paired ablation study across two conditions (300 runs, 150 per condition) confirms that CFR rises from 0.0% to 38.0% when the System-3/0 architecture is removed ($p < 0.001$, Cohen's $h = -1.328$), while mean token expenditure drops by 3.6×, consistent with the depth-k composability result (Theorem 3.2).

9.2 Broader Implications

The central engineering takeaway is practical: unreliable model outputs can still be composed reliably if every boundary enforces typed syntactic contracts. Any multi-agent pipeline that chains free-form outputs without such boundaries remains vulnerable to silent cascades. The System-3/0 design provides one concrete way to enforce those boundaries while retaining architectural flexibility.

The MAS-OS extension (Section 7) demonstrates that the framework naturally maps to operating-system abstractions (goals as processes, the registry as a process table, System-3 as a scheduler, and System-0 as a protection ring), suggesting that the next generation of agentic platforms may benefit from adopting OS-level design principles rather than treating agents as loosely coupled prompt chains.

9.3 Closing Remarks

Multi-agent systems are moving from demos to real deployments, but reliability engineering still lags behind model capability. We argue that closing that gap requires stronger *systems* design: explicit contracts, budget-aware orchestration, and deterministic validation boundaries. The System-3/0 Blueprint is one implementation of that direction.

Acknowledgements

I thank my supervisors, Dr. Anand Rao (Carnegie Mellon University) and Dr. Chittaranjan Hota (BITS Pilani, Hyderabad Campus), for their feedback throughout this work; the Department of Computer Science and Information Systems at BITS Pilani, Hyderabad Campus; and the open-source communities behind Ollama, LangGraph, LangChain, and Outlines. The reference implementation and evaluation harness are available at `https://github.com/Jyotirmoy17/System-3-0-MAS-OS`.

Bibliography

[Ant24] Anthropic. *Model Context Protocol (MCP)*. `https://modelcontextprotocol.io`. 2024 (cit. on p. 6).

[APS11] Y. Abbasi-Yadkori, D. Pál, and C. Szepesvári. „Improved Algorithms for Linear Stochastic Bandits“. In: *Advances in Neural Information Processing Systems*. Vol. 24. 2011 (cit. on p. 7).

[BFV24] L. Beurer-Kellner, M. Fischer, and M. Vechev. *Guiding LLMs the Right Way: Fast, Non-Invasive Constrained Generation*. 2024. arXiv: `2403.06988` (cit. on p. 8).

[Bha26] A. Bhardwaj. *Agent Behavioral Contracts*. 2026. arXiv: `2602.22302` (cit. on p. 8).

[CZZ24] L. Chen, M. Zaharia, and J. Zou. „FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance“. In: *Transactions on Machine Learning Research* (2024) (cit. on p. 6).

[DD25] M. Dewes and R. Dimitrova. „Contract-based Design and Verification of MAS“. In: *AAAI Conference on Artificial Intelligence*. 2025 (cit. on p. 8).

[Dek+25] J. Dekoninck et al. „A Unified Approach to Routing and Cascading for LLMs“. In: *International Conference on Machine Learning*. 2025 (cit. on p. 6).

[Din+24] D. Ding et al. „Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing“. In: *International Conference on Learning Representations*. 2024 (cit. on p. 6).

[Don+24] Y. Dong et al. „XGrammar: Flexible and Efficient Structured Generation“. In: *MLSys*. 2024 (cit. on p. 8).

[Ekb+26] C. Ekbote et al. *Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets*. 2026. arXiv: `2604.02460` (cit. on p. 9).

[Goo25] Google. *Agent-to-Agent (A2A) Protocol*. `https://google.github.io/A2A`. 2025 (cit. on p. 6).

[Hon+24] S. Hong et al. „MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework“. In: *International Conference on Learning Representations*. 2024 (cit. on p. 7).

[Kha+24] O. Khattab et al. „DSPy: Compiling Declarative Language Model Calls“. In: *International Conference on Learning Representations*. 2024 (cit. on p. 7).

[Kim+25] S. Kim et al. *Towards a Science of Scaling Agent Systems*. 2025. arXiv: `2512.08296` (cit. on p. 9).

[Mei+24] K. Mei et al. *AIOS: LLM Agent Operating System*. Preprint. 2024 (cit. on p. 9).

[Mik+25] R. Mikulski et al. *Towards Assume-Guarantee Verification of Abilities in Stochastic MAS*. 2025 (cit. on p. 8).

[Ong+25] I. Ong et al. „RouteLLM: Learning to Route LLMs with Preference Data“. In: *International Conference on Learning Representations*. 2025 (cit. on p. 6).

[Pac+23] C. Packer et al. *MemGPT: Towards LLMs as Operating Systems*. Preprint. 2023 (cit. on p. 9).

[Par+24] J. Park et al. „Grammar-Aligned Decoding“. In: *Advances in Neural Information Processing Systems*. 2024 (cit. on p. 8).

[Pat+23] S. Patil et al. *Gorilla: Large Language Model Connected with Massive APIs*. Preprint. 2023 (cit. on p. 9).

[Sch+23] T. Schick et al. *Toolformer: Language Models Can Teach Themselves to Use Tools*. Preprint. 2023 (cit. on p. 9).

[Su+25] Y. Su et al. *CP-Router: Conformal Prediction for Uncertainty-Aware LLM Routing*. Preprint. 2025 (cit. on p. 6).

[Wei78] M. Weitzman. *Optimal Search for the Best Alternative*. Tech. rep. Department of Energy, 1978 (cit. on p. 7).

[WL23] B. T. Willard and R. Louf. *Efficient Guided Generation for LLMs*. Outlines Foundation. 2023 (cit. on p. 8).

[Wu+24] Q. Wu et al. „AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation“. In: *International Conference on Learning Representations*. 2024 (cit. on p. 7).

[Xie+24] T. Xie et al. *OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments*. Preprint. 2024 (cit. on p. 9).

[YT26] Q. Ye and J. Tan. *Agent Contracts: A Formal Framework for Resource-Bounded Autonomous AI Systems*. 2026. arXiv: 2601.08815 (cit. on p. 8).

[Zho+23] S. Zhou et al. „WebArena: A Realistic Web Environment for Building Autonomous Agents“. In: *International Conference on Learning Representations*. 2023 (cit. on p. 9).